International Application Rating Algorithm

Final Report

Washington State University – International Programs

We bring WSU to the world, and the world to WSU.

WSU International

Mentor:

Prateek Kumar

Admissions Intelligence Group (AIG)

Khushi Panchal

Steven Bennett

Cpt_S 423 Software Design Project II

Spring 2026

TABLE OF CONTENTS

I. Introduction **3**

II. Team Members & Bios **4**

III. Project Requirements Specification **5**

III.1. Project Stakeholders 5

III.2. Use Cases 6

III.3. Functional Requirements 9

III.4. Non-Functional Requirements 12

III.5. Standards 13

IV. Software Design - From Solution Approach **14**

IV.1. Architecture Design 14

IV.2. Data design 17

IV.3. User Interface Design 17

IV.4. Constraints 21

V. Test Case Specifications and Results **23**

V.1. Testing Overview 23

V.2. Environment Requirements 25

V.3. Test Results 26

VI. Projects and Tools used **27**

VII. Description of Final Prototype **29**

VIII. Social Responsibility and Broader Impacts **29**

IX. Product Delivery Status **34**

X. Conclusions and Future Work Error! Bookmark not defined.

X.1. Limitations and Recommendations **Error! Bookmark not defined.**

X.2. Future Work **Error! Bookmark not defined.**

XI. Acknowledgements Error! Bookmark not defined.

XII. Glossary Error! Bookmark not defined.

XIII. References Error! Bookmark not defined.

XIV. Appendix A – Team Information Error! Bookmark not defined.

XV. Appendix B - Example Testing Strategy Reporting Error! Bookmark not defined.

XVI. Appendix C - Project Management Error! Bookmark not defined.

I. Introduction

The **International Applicant Rating Algorithm (IARA)** is a decision-support system developed for Washington State University (WSU) to enhance the efficiency and consistency of international student application evaluations. The current process within the International Programs Office relies heavily on manual review of Slate data, including GPA, curriculum rigor, travel history, financial documentation, English proficiency, and applicant essays. This approach is time-consuming, labor-intensive, and susceptible to inconsistencies across evaluators, particularly during peak application cycles.

To address these challenges, IARA automates critical components of the evaluation process while preserving transparency, fairness, and counselor oversight. The system generates categorical scores, computes a composite rating, and provides detailed justifications for each evaluation. It also incorporates AI-driven essay analysis, with a fallback mechanism when external services are unavailable. By standardizing evaluation criteria and reducing manual workload, IARA facilitates a more efficient, consistent, and reliable admissions workflow.

The need for such systems has grown as universities manage increasingly large and diverse international applicant pools. Existing solutions typically focus on isolated tasks such as grade conversion, language proficiency assessment, or document verification, offering limited integration across the admissions pipeline. Meanwhile, advances in natural language processing (NLP) and optical character recognition (OCR) have demonstrated the potential to automate tasks such as essay evaluation and document validation. However, few systems integrate these capabilities into a cohesive platform tailored specifically to international admissions.

IARA addresses this gap through modular architecture that integrates applicant scoring, essay analysis, fraud detection, and batch processing. The system leverages NLP models to evaluate written responses and OCR-based tools to identify inconsistencies in submitted documents, enabling more comprehensive and accurate assessments. A user-friendly web interface further supports adoption by minimizing training requirements and streamlining counselor workflows.

Developed in collaboration with Washington State University's International Programs Office, IARA is designed to align with key stakeholder priorities, including transparency, consistency, and auditability. The system supports admissions counselors, administrative staff, and institutional leadership while maintaining compliance with regulatory standards such as FERPA and GDPR. Ultimately, IARA contributes to a more equitable, scalable, and data-driven approach to international admissions evaluation.

II. Team Members & Bios

Khushi Panchal is a senior at Washington State University, graduating in Spring 2026 with a Bachelor of Science in Computer Science. She is interested in data analytics, web development, and software engineering, with experience in full-stack applications and data-driven systems. Her technical skills include Python, JavaScript, HTML, CSS, and familiarity with cloud platforms such as AWS and Google Cloud. For the International Applicant Rating Algorithm (IARA) project, Khushi served as the lead full-stack developer across both semesters. She was responsible for developing the rubric-based scoring engine, batch processing and CSV import/export functionality, and the overall Flask application architecture. She also expanded the database layer with automatic schema migration, implemented report generation using ReportLab, and led a complete frontend redesign aligned with WSU standards. Additionally, she developed the admin user management system and contributed extensively to project documentation.

Steven Bennett is a Software Engineering student with interests in AI-assisted systems, backend development, and applied machine learning. His technical skills include Python, Flask, SQL, Java, and OpenAI-based reasoning tools, along with experience in OCR and computer vision pipelines. For the IARA project, Steven developed the Financial and Transcript Fraud Detection subsystems. His work included implementing Flask routes, building OCR extraction pipelines, designing heuristic-based image analysis, and integrating AI-assisted anomaly detection. In later development phases, he added real-time progress indicators for fraud detection, developed an automated test suite for both subsystems, resolved issues in the PDF export pipeline, and contributed to system integration and stakeholder communication.

III. Project Requirements Specification

This section presents the functional and non-functional requirements, stakeholders, and use cases for the *International Applicant Rating Algorithm (IARA)* system. It defines the system's intended behavior, key features, and interactions with admissions counselors and other stakeholders involved in evaluating international applicants [1]. The goal is to ensure a transparent, efficient, and consistent evaluation process aligned with institutional and regulatory standards.

III.1. Project Stakeholders

Our primary stakeholder is Trevor Kingsley of Washington State University's Office of International Programs, who serves as the client for the IARA system. The client requires a system that ensures transparency in scoring decisions, consistency in applicant evaluation, reduced manual workload for admissions counselors, and the ability to justify decisions during audits or appeals, while maintaining counselor judgment in the process.

Admissions counselors are the primary end users and require an accurate, intuitive interface that minimizes errors and clearly presents scoring results, along with explainable outputs and efficient report generation.

Administrative staff and university officials are secondary stakeholders who require user management, system monitoring, and compliance with regulations such as FERPA and GDPR, ensuring secure and auditable handling of applicant data.

Although international applicants are not direct users, they benefit from a more consistent and equitable evaluation process. Future developers are also stakeholders, requiring maintainable code and documentation to support ongoing development and integration.

III.2. Use Cases

The following use-cases describe common interaction scenarios between users and the system and demonstrate the application of functional requirements. These interactions are illustrated in the use case diagram shown in Figure 1 [2].

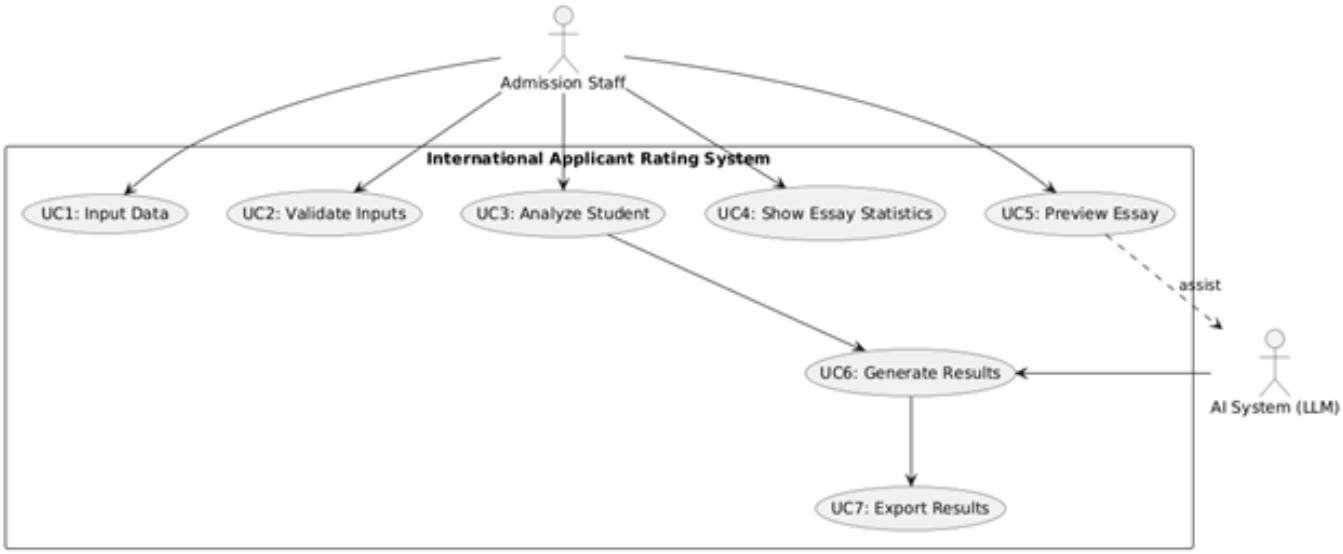

Figure 1: Use Case Diagram

Use Case 1: Input Student Profile Data

Actor	Admissions Counselor
Precondition	Counselor is logged in and form is open and empty
Postcondition	Student academic, financial, and behavioral attributes are entered into the system
Main Flow	Counselor enters Student ID, GPA, curriculum type, country, travel history, essay, risk factors → submits
Alternative Flow	Missing/invalid fields → form shows validation errors
Related Requirements	FR-01, FR-02

Use Case 2: Analyze Student [3]

Actor	Admissions Counselor
Precondition	Student profile is entered and complete
Postcondition	The system displays positive, negative, and composite scores with AI insights.
Main Flow	Counselor clicks “Analyze” → system validates fields → calculates scores → displays factor breakdown
Alternative Flow	AI timeout → fallback used; invalid config → process aborted.
Related Requirements	FR-03, FR-04, FR-05, FR-06, FR-07, FR-08, FR-10

Use Case 3: Show Essay Statistics

Actor	Admissions Counselor
Precondition	Essay text is present
Postcondition	Panel displays essay statistics (characters, words, reading time)
Main Flow	Counselor clicks “Stats” → system calculates and displays stats
Alternative Flow	Essay empty → no stats shown
Related Requirements	FR-02, FR-10

Use Case 4: Preview Essay

Actor	Admissions Counselor
Precondition	Essay text is present
Postcondition	Formatted, read-only preview is shown
Main Flow	Counselor clicks “Preview” → system formats essay and displays
Alternative Flow	Essay empty → prompt for input
Related Requirements	FR – 02

Use Case 5: Clear Essay

Actor	Admissions Counselor
Precondition	Essay section contains text
Postcondition	Essay field is reset to empty
Main Flow	Counselor clicks “Clear” → essay cleared
Alternative Flow	Essay already empty → no change
Related Requirements	FR – 01, FR – 02

Use Case 6: Apply Risk Factors

Actor	Admissions Counselor
Precondition	Risk factor checkboxes visible
Postcondition	Selected risk factors applied during scoring
Main Flow	Counselor selects risk factors → system applies during Analyze Student
Alternative Flow	No risk factors selected → system defaults to low risk
Related Requirements	FR-04, FR-06, FR-07

Use Case 7: Generate Reports

Actor	Admissions Counselor
Precondition	Student analysis has been completed.
Postcondition	A formatted report is generated and downloaded
Main Flow	Counselor enters optional comments → the system generates and downloads the report.
Alternative Flow	If a database error occurs, the system displays an error message and does not save the report.
Related Requirements	FR-12

Use Case 8: Manage Users

Actor	Administrator
Precondition	The user is logged in with administrative privileges.
Postcondition	User accounts are created, deactivated, or deleted.
Main Flow	Administrator accesses the user management interface → admin performs account management actions → the system logs all actions.
Alternative Flow	If an attempt is made to delete a protected administrator account, the system rejects the action and displays a warning message.
Related Requirements	FR-11

Use Case 9: Screen Financial or Transcript document

Actor	Admissions Counselor
Precondition	A valid document is available for upload.
Postcondition	The system displays fraud detection indicators and a document-level risk classification.
Main Flow	Counselor uploads a document → system performs OCR, heuristic checks, and AI-based analysis → results are displayed
Alternative Flow	If the file type is unsupported, the system displays an error message.
Related Requirements	FR-13

III.3. Functional Requirements

This section defines the functional requirements of the system, including descriptions, sources, and priority levels. These requirements specify the expected behavior of both the frontend and backend components.

III.3.1. Student Data Input

Input Student Data

Description	The system shall enable authorized users to input comprehensive student information, including Student ID, GPA, country of origin, curriculum type, travel history, personal essay, and associated risk factors
Source	Client rubric, Team brainstorm
Priority	Level 0

Field Validation

Description	The system shall validate required fields and provide real-time feedback if any inputs are missing.
Source	Client rubric, Team brainstorming
Priority	Level 0

III.3.2. Essay Analysis

Essay Analysis with Fallback

Description	The system shall perform AI-driven essay analysis to assess sentiment, motivation, and academic focus, with fallback to local heuristic analysis in the event of API failure.
Source	Client rubric, GenAI brainstorming
Priority	Level 0

AI Insights

Description	The system shall show AI insights and recommendations based on final scoring.
Source	Client rubric, AI brainstorming
Priority	Level 1

AI Fallback Support

Description	The system shall support both AI-powered analysis and local fallback analysis when APIs are unavailable.
Source	Client rubric, Team brainstorming, GenAI brainstorming
Priority	Level 0

III.3.3. Scoring Engine

Positive Score Calculation

Description	The system shall calculate positive scores based on GPA, curriculum rigor, travel experience, essay quality, and motivational indicators derived from AI analysis.
Source	Client rubric
Priority	Level 0

Negative Score Calculation

Description	The system shall calculate negative deductions based on fee waiver requests, payment issues, pending documents, and early I-20 requests.
Source	Client rubric
Priority	Level 0

Composite Score Calculation (POS/NEG/AI)

Description	The system shall compute a final composite score combining positive scores, negative deductions, and AI analysis.
Source	Client rubric
Priority	Level 0

Financial Risk Assessment

Description	The system shall assess financial risk levels using application fee waivers, payment issues, and regional risk factors
Source	Client rubric
Priority	Level 0

III.3.4. Results and System Features

Results Presentation

Description	The system shall present results in a structured format, including academic, financial, and behavioral scores, overall enrollment probability, and a detailed factor breakdown.
Source	Client rubric
Priority	Level 0

Admin User Management

Description	The system shall enable administrators to create, deactivate, and delete user accounts, with all actions recorded in an activity log.
Source	Team design, Sprint 5
Priority	Level 1

Report Generation

Description	The system shall generate downloadable PDF and text reports that include scores, essay metrics, staff comments, fraud analysis, and recommendations.
Source	Client rubric
Priority	Level 0

Fraud Detection Progress Feedback

Description	The system shall display real-time progress indicators during fraud detection processing.
Source	Usability review, Sprint 5
Priority	Level 1

Fraud Detection Testing

Description	The system shall include automated testing to validate fraud detection logic for financial and transcript documents.
Source	Team design, Sprint 5
Priority	Level 1

III.4. Non-Functional Requirements

The non-functional requirements define the system's operational qualities, including performance, scalability, security, and usability. These requirements ensure that the system meets quality standards beyond its core functionality. The details of the non-functional requirements are outlined below.

Non-Functional Requirement	Description
Scalability	The system shall support up to 50 concurrent users and handle large applicant volumes through batch processing and parallel computation without performance degradation.
Data Accuracy	The system shall ensure accurate processing of student data and flag any discrepancies against the client-provided dataset.
Security and Role-Based Access [6]	The system shall protect sensitive data through secure authentication and role-based access while complying with regulations such as FERPA and GDPR, including audit logging of user activity.
Compliance and Auditability [7]	The system shall maintain at least 99% uptime and ensure continuous operation through fallback mechanisms when AI services are unavailable.
Availability	The system shall maintain at least 99% uptime, with downtime limited to scheduled maintenance that is communicated at least 24 hours in advance.
Performance	The system shall efficiently handle large volumes of applicants by supporting batch processing and parallel computation for scoring operations.
Data Storage and Export	The system shall support persistent storage and provide configurable export formats, including PDF, to accommodate reporting and printing needs.
Logging and Monitoring	The system shall track key performance metrics (e.g., processing time, error rates, AI response time) and maintain detailed logs including timestamps, user activity, and system configuration.
Accessibility	The system shall provide an accessible user interface that supports users with disabilities, ensuring clear error messages and efficient completion of core tasks.
Reliability (AI Fallback Handling)	The system shall maintain reliable operation by providing fallback processing when AI services are unavailable or exceed timeout thresholds, clearly indicating fallback usage.

III.5. Standards

The *International Applicant Rating Algorithm* (IARA) adheres to established technical, regulatory, and ethical standards to ensure system quality, accessibility, and secure handling of applicant data. These standards guide the development of system requirements, user interface design, and data protection practices.

Standard / Code	Domain	Description	Application in IARA
IEEE 830 - Software Requirements Specification (1998)	Software Documentation	Defines structure and content for software requirements and specifications	Used to organize functional and non-functional requirements, use cases, and the traceability matrix
W3C WCAG 2.2 - Web Content Accessibility Guidelines	Web / UI Design	Ensures accessibility for users with visual or motor impairments	Applied contrast checks, alt text, and keyboard navigation for counselor interface
GDPR (2018) - General Data Protection Regulation	Data Privacy	Regulates the collection and storage of personal data	Implemented anonymized data storage and consent prompts before student information submission
FERPA (1974) - Family Educational Rights and Privacy Act	Educational Data Protection	Protects the confidentiality of student records	Ensure that applicant data is accessible only to authorized users
IEEE 1016 - Software Design Description	Design Documentation	Provides standards for clear system design and documentation	Applied to system design artifacts, including class diagrams and use case documentation
ACM Code of Ethics	Professional Conduct	Promotes fairness, accountability, and ethical computing practices	Followed in AI-based scoring to reduce bias and ensure transparency in evaluations
WSU Brand Identity Standards	UI / Brand Compliance	Defines official university design elements such as color, typography, and layout	Applied consistently across the interface using WSU-approved design guidelines

IV. Software Design - From Solution Approach

This section provides a detailed solution approach for the *International Applicant Rating Algorithm* (IARA) system, designed to support structured and consistent evaluation of international applicants using rubric-based scoring, AI-assisted analysis, and document verification. The purpose of this section is to outline the architectural decisions, system components, and strategies implemented to meet the project requirements. The intended audience includes developers, technical stakeholders, and evaluators involved in developing and reviewing the system.

The IARA system is designed to process applicant data, compute rubric-based scores, and generate structured evaluation results for admissions counselors. It integrates multiple subsystems, including a scoring engine, an NLP-based essay analysis module, and a document fraud detection module, to support comprehensive applicant assessment.

At its core, the system follows a layered architecture that separates presentation, application logic, service processing, report generation, and data storage. This structure enables efficient communication between components while maintaining modularity and scalability.

The system consists of several key components: a Flask-based frontend for user interaction, a backend application layer for request handling and orchestration, a scoring engine for applicant evaluation, an NLP/essay analysis handler for generating insights from text, a document fraud detection handler for analyzing uploaded files, a report generation module for producing PDF reports, and a database layer for persistent storage of application data, logs, and results

IV.1. Architecture Design

In this section, we describe the system architecture and its components, including the interfaces used for interaction between them. The architecture provides a high-level design view that supports implementation and further system development.

IV.1.1. Overview

The IARA system follows a layered web application architecture that separates the system into presentation, application logic, service processing, report generation, and data storage layers.

The system uses a Flask-based web interface that allows admissions counselors to input applicant data, upload documents, and view scoring results. The Flask application (app.py) acts as the central controller, handling requests and coordinating interactions between system components.

The Service Layer contains the core analytical components. The scoring engine computes rubric-based academic, financial, and behavioral scores. The NLP/essay analysis module evaluates applicant essays and generates structured insights. The document fraud detection module processes uploaded financial and transcript documents using OCR, heuristic checks, and AI-based analysis.

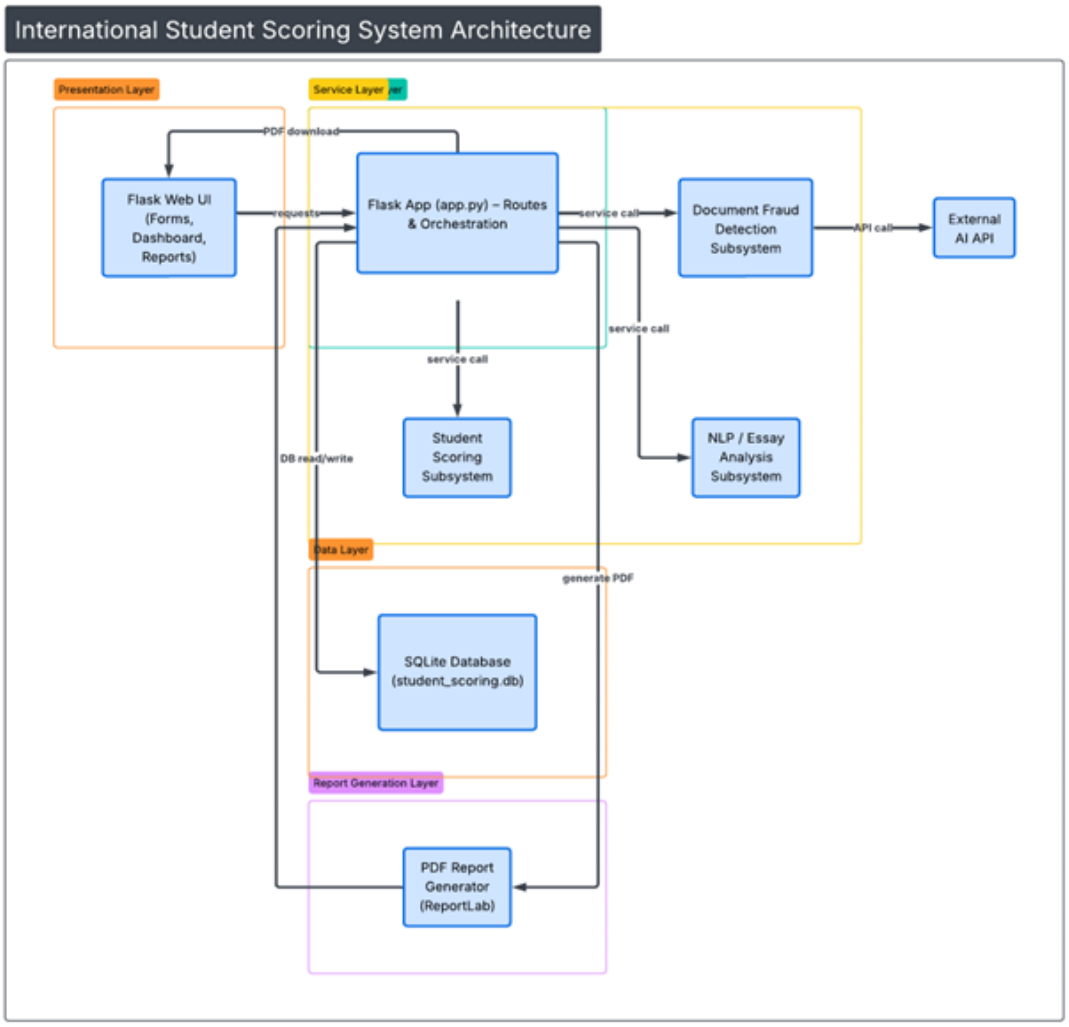

Figure 2: Architecture Diagram

The Report Generation Layer produces downloadable PDF reports containing scoring results and analysis outputs. The Data Layer stores persistent information, including applicant records, logs, and generated reports using a SQLite database.

This layered design ensures separation of concerns, modularity, and scalability, enabling efficient processing and reliable system performance.

IV.1.2. Subsystem Decomposition

The system is divided into smaller, manageable subsystems to streamline development and integration. Each subsystem focuses on a specific set of responsibilities to ensure modularity and maintainability.

IV.1.2.1. Web Interface Subsystems

The Web Interface Subsystem provides the user-facing layer of the system and was fully redesigned in Sprint 5 using the official WSU design system CSS bundle. It standardizes layout across all eleven templates, including a consistent header, navigation bar, utility bar, and footer. The login page uses a two-panel layout. Due to styling conflicts with the WSU CSS reset, custom SVG-based risk factor checkboxes were implemented. File upload components were refactored using hidden inputs with explicit click delegation to resolve browser inconsistencies. An administrative interface at /admin/users supports user management with role-based access control and a protected default admin account.

IV.1.2.2. Application Orchestration Subsystem

The Application Orchestration Subsystem is implemented in app.py and serves as the central controller of the system. It manages routing, input validation, and coordination between all major subsystems, including scoring, NLP analysis, fraud detection, and report generation. It also handles database interactions and determines whether responses are returned as JSON or rendered templates. This centralization ensures that analytical subsystems remain modular and decoupled.

IV.1.2.3. Student Scoring Subsystem

The Student Scoring Subsystem is implemented in student_analyzer.py and is responsible for rubric-based evaluation of applicant data. It calculates positive scores based on GPA, curriculum type, and travel history, and applies deductions based on identified risk factors. The subsystem produces a final composite score and categorization (e.g., Strong Admit, Admit, Review, Reject), along with a factor-level breakdown for interpretability. A safety wrapper ensures consistent and front-end safe output formatting.

IV.1.2.4. NLP / Essay Analysis Subsystem

The NLP / Essay Analysis Subsystem processes applicant essays using transformer-based models. It performs sentiment analysis, computes similarity embeddings, and generates structured essay evaluation metrics. The subsystem outputs standardized results that are integrated into both the dashboard and report generation modules. Model inference and processing logic are encapsulated to allow future model replacement without affecting other system components.

IV.1.2.5. Document Fraud Detection Subsystems

The Document Fraud Detection Subsystem analyzes uploaded financial and transcript documents using OCR, heuristic validation, and AI-based anomaly detection. In Sprint 5, a real-time progress indicator was added to improve user feedback during multi-page processing. An automated test suite was implemented for validation, and a PDF export issue causing text overflow was resolved to ensure correct report formatting.

IV.1.2.6. Report Generation Subsystem

The Report Generation Subsystem is implemented in repor_generator.py and generates formal PDF reports summarizing applicant evaluations. Reports include scoring results, essay metrics, fraud detection outputs, and reviewer comments. The subsystem uses ReportLab to produce structured documents and returns reports as in-memory byte streams for download or storage. Separation of report logic from analytical components improves maintainability.

IV.1.2.7. Data Persistence Subsystem

The Data Persistence Subsystem manages all database operations and was extended in Sprint 5 with additional administrative and reporting capabilities. New methods include user and report retrieval, dashboard statistics, activity logging, and enhanced user creation with role and profile support. A database migration function automatically resolves schema inconsistencies at startup. The system uses salted PBKDF2 hashing for password security and operates in Write-Ahead Logging (WAL) mode to support safe concurrent access.

IV.2. Data design

The IARA system uses structured datasets, internal data objects, and persistent storage to support applicant evaluation, analysis, and reporting. Primary data includes applicant information such as GPA, curriculum type, country, travel history, essays, and risk factors, which are stored in a SQLite database along with user accounts, logs, and generated reports.

During processing, data is transformed into internal Python structures such as dictionaries and objects representing applicant profiles and scoring results. Temporary data structures are used for intermediate NLP and fraud detection outputs, including tokenized text, extracted features, and OCR results.

Essay analysis generates embeddings using transformer models, which are temporarily stored in memory for similarity calculations and insight generation. Fraud detection outputs are also structured before being stored or exported. Overall, the data design supports efficient flow between input, processing, and storage layers.

IV.3. User Interface Design

The user interface was designed to support a structured, transparent, and efficient applicant evaluation workflow. The design follows a modular, task-oriented layout rather than a conversational interface, with the objective of ensuring clarity, consistency, and traceability across all stages of the evaluation process. In particular, the interface prioritizes explainability of automated scoring outputs and maintains human oversight in all decision-making components.

The interface is organized into four primary functional modules: applicant input, evaluation results, fraud screening, and report generation. This separation of concerns reduces cognitive load by allowing users to engage with one component at a time while preserving a coherent

end-to-end workflow. Each module is designed to ensure that user actions are directly mapped to backend logic and stored evaluation structures.

The applicant input form is designed to enforce structured data entry aligned with the underlying scoring rubric. Input controls such as dropdown menus and predefined selection fields are used for attributes including GPA range, curriculum type, travel history category, and risk factors. This design minimizes ambiguity associated with free-text input and ensures consistency between frontend inputs and backend scoring logic. Essay responses are captured using a constrained text input field that supports sufficient length while maintaining structured form design principles.

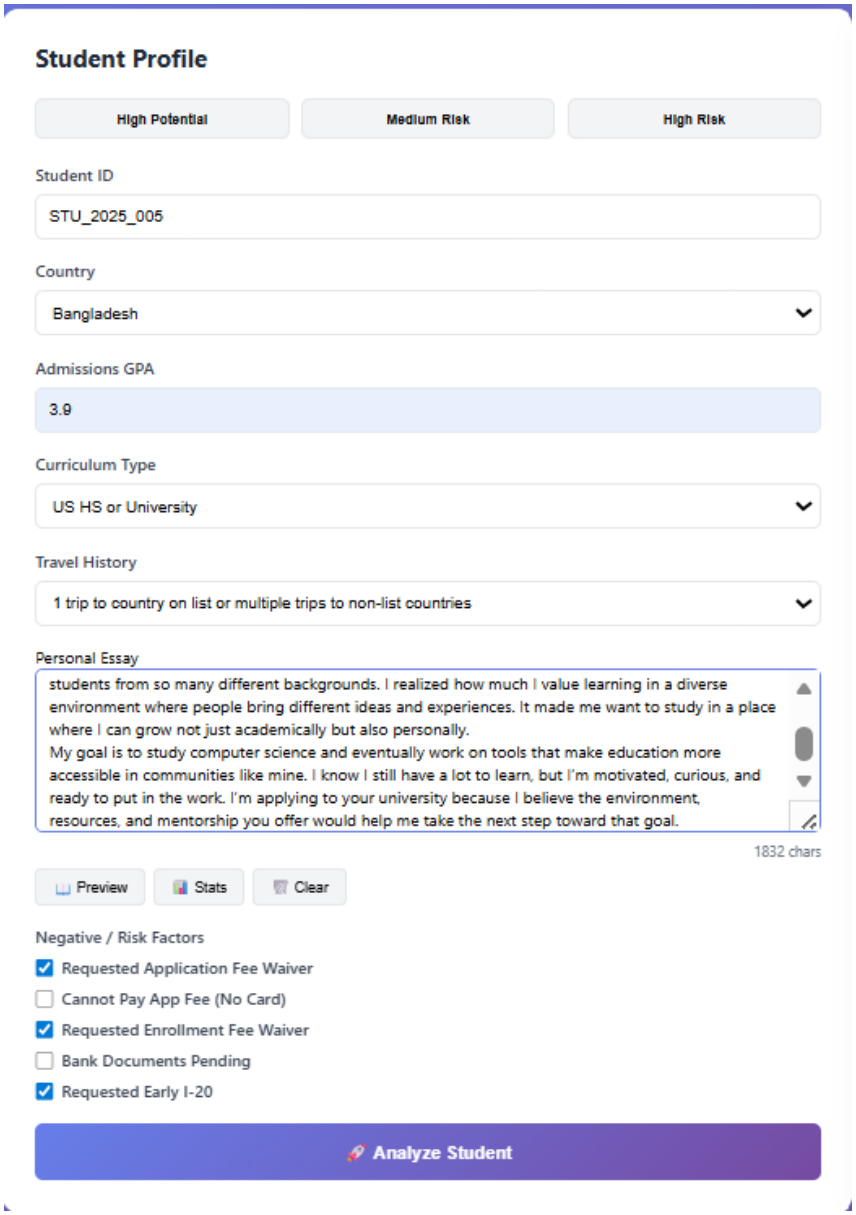

Figure 3: Input Form Design

The results dashboard presents computed evaluation outcomes in a decomposed format to enhance interpretability. Rather than displaying only a final score, the interface provides a breakdown of positive contributions, negative deductions, the aggregated score, and the corresponding recommendation category. This explicit separation of components improves transparency and supports accountable decision-making by making the scoring process observable and verifiable.

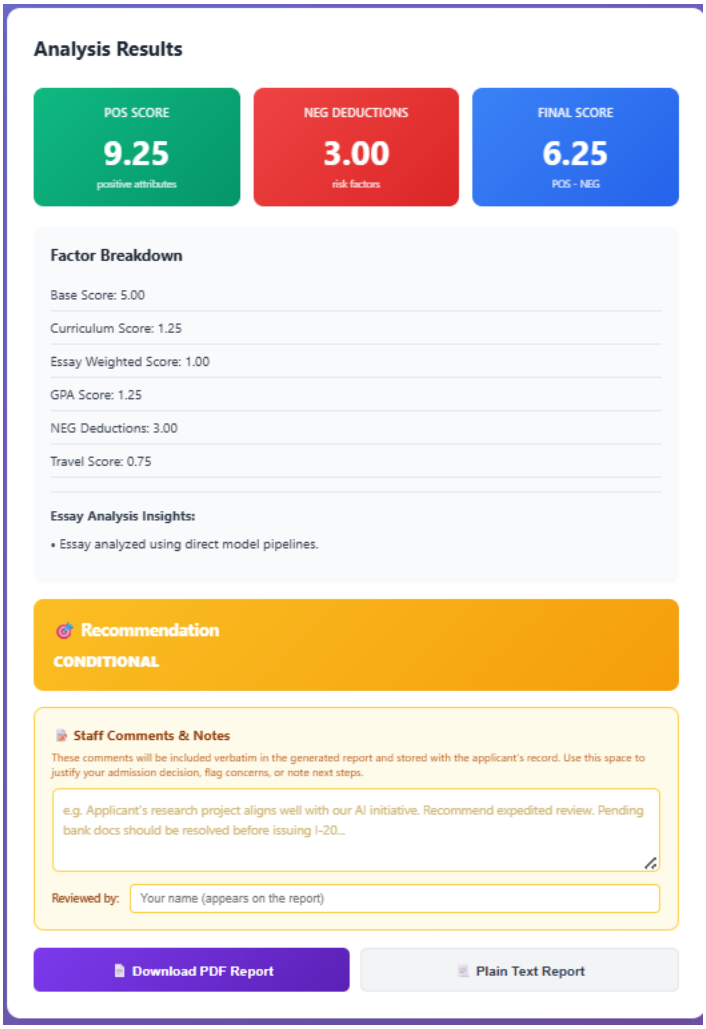

Figure 4: Result Dashboard

The fraud screening module is designed to support human-in-the-loop document assessment. Users may upload supporting documents in PDF format, after which the system presents detected fraud signals, severity classifications (LOW, MEDIUM, HIGH), and associated confidence scores. The interface avoids binary fraud classification in favor of evidence-based indicators, thereby ensuring that automated outputs function as decision-support tools rather than autonomous decision mechanisms.

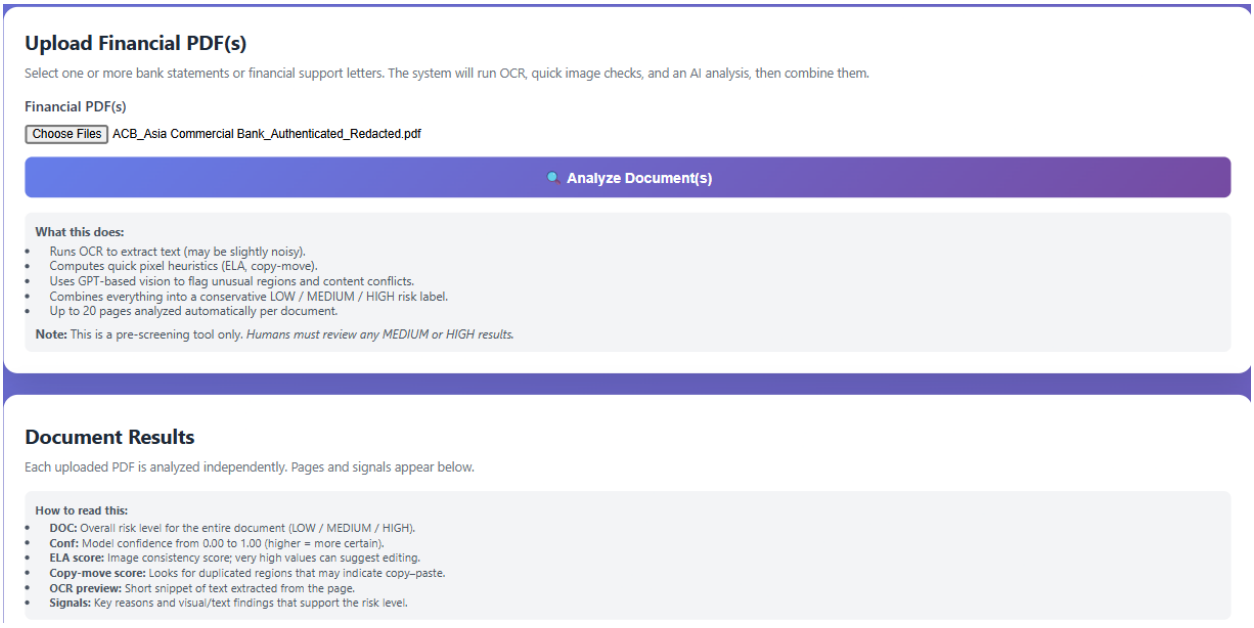

Figure 5: Fraud Screening Interface

The report generation module is integrated directly into the evaluation workflow to ensure continuity between analysis and documentation. After reviewing applicant scores and fraud screening outputs, users may add contextual comments and generate a standardized PDF report. The generated report is derived from the same structured data used in the evaluation dashboard, ensuring consistency, reproducibility, and auditability of recorded decisions.

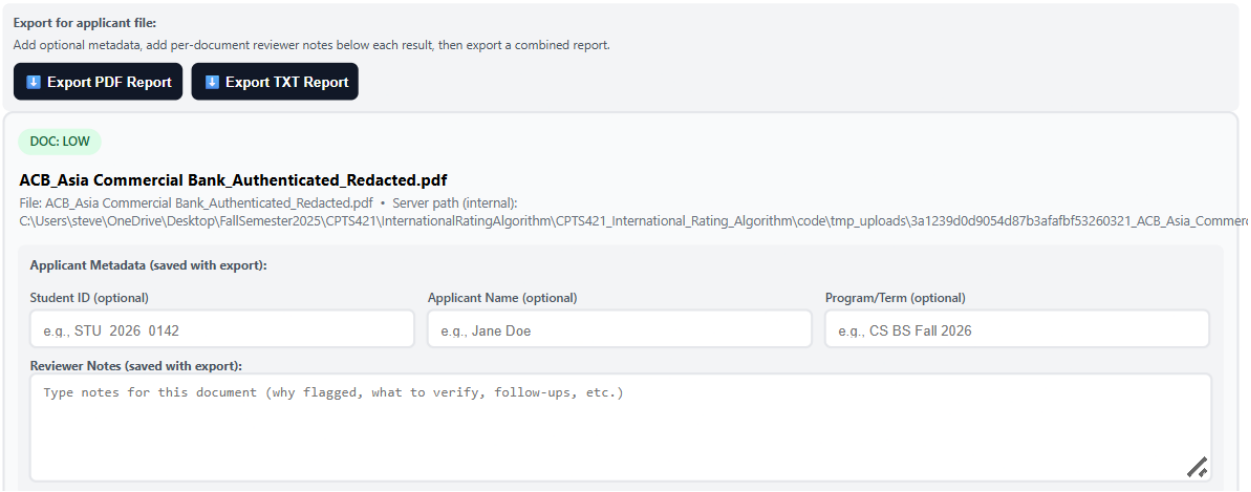

Figure 6: Report Generation Interface

IV.4. Constraints

Software engineering design is an iterative decision-making process in which system requirements are satisfied under a set of technical, operational, and contextual constraints. In this project, multiple constraints influenced architectural decisions, system implementation, and interface design. These constraints required trade-offs between performance, scalability, usability, and system complexity.

IV.4.1. Technical Constraints

The system was implemented using Python with the Flask web framework, which constrains the architecture to a monolithic or server-rendered web application model rather than a distributed microservices architecture. SQLite was selected as the primary database due to its simplicity and ease of deployment in a prototype environment; however, this introduces limitations in scalability, concurrency handling, and performance under high write loads compared to enterprise-grade database systems.

The natural language processing (NLP) and document analysis components rely on a combination of locally hosted transformer-based models and external application programming interfaces (APIs). This introduces computational constraints related to memory usage, inference latency, and processing overhead, particularly in document-heavy fraud detection workflows involving optical character recognition (OCR) and image-based analysis. To mitigate performance limitations, document processing is restricted to a predefined page limit per upload, ensuring predictable runtime behavior and manageable resource consumption.

IV.4.2. Security and Privacy Constraints

The system processes sensitive applicant information, including academic records, financial documents, and personal statements. Consequently, security and privacy requirements significantly influenced system design decisions.

Authentication is implemented using salted PBKDF2 hashing to prevent storage of plaintext credentials. Additionally, role-based access control mechanisms are used to restrict system functionality based on user roles (e.g., administrator and reviewer). Uploaded documents are processed locally and stored as binary large objects (BLOBs) within the SQLite database to reduce external data exposure.

However, integration with external AI-based APIs introduces privacy constraints related to third-party data transmission and compliance considerations. As a result, the system is designed as a decision-support tool rather than a fully automated decision-making system, ensuring that human oversight remains central in high-risk fraud evaluation scenarios.

IV.4.3. Usability Constraints

The system is designed for admissions staff who are non-technical users; therefore, usability constraints strongly influenced interface design. The interface prioritizes clarity, structured input, and interpretability over configurability.

To minimize user error and cognitive load, the system employs dropdown menus, predefined categorical inputs, and standardized scoring fields instead of free-text or unrestricted numerical inputs. Risk indicators are represented using clearly defined severity labels (LOW, MEDIUM, HIGH) to ensure consistent interpretation across users. Additionally, visual score breakdowns are used to improve interpretability of evaluation outcomes while maintaining alignment with the underlying scoring rubric.

A further usability constraint arises from the need to balance detailed analytical output with readability. Fraud detection explanations must provide sufficient transparency for review while avoiding excessive technical complexity that may hinder decision-making efficiency.

IV.4.4. Extensibility Constraints

The system architecture is modular, with distinct subsystems for scoring, natural language processing, fraud detection, reporting, and database management. This separation of concerns was intentionally adopted to support future extensibility and maintainability.

Although the current implementation is deployed as a single Flask-based application, this design choice prioritizes simplicity over horizontal scalability. Transitioning to a distributed architecture would require refactoring of inter-module communication and service boundaries.

Model-level constraints also exist due to reliance on specific transformer models and external APIs. Changes in model versions or API behavior may affect scoring consistency and output stability; therefore, version control and regression testing are necessary for long-term reliability. Similarly, while SQLite is suitable for prototyping and low-concurrency environments, it imposes limitations on scalability that would need to be addressed in production deployment.

Overall, the system design reflects deliberate trade-offs between simplicity, interpretability, and extensibility, ensuring that the core functionality is robust while remaining adaptable to future enhancements.

V. Test Case Specifications and Results

This section outlines the testing strategy and evaluation of the International Student Scoring System. The objective of testing is to ensure that the system meets functional and non-functional requirements, operates reliably under expected usage conditions, and correctly integrates all subsystems into a unified workflow. Testing includes both automated unit-level validation and manual end-to-end system verification across scoring, NLP analysis, fraud detection, reporting, and data persistence components.

V.1. Testing Overview

The primary objective of testing the International Student Scoring System is to verify that all core functionalities operate correctly both in isolation and as part of the complete system workflow. The testing process focuses on validating rubric-based score computation, risk deduction logic, NLP-based essay analysis, OCR-driven fraud detection, severity classification (LOW / MEDIUM / HIGH), and report generation accuracy.

Additional objectives include verifying correct handling of reviewer comments, ensuring accurate export of analysis results, and validating persistence of records in the underlying database. Testing also ensures that the Flask-based application successfully orchestrates interactions between scoring, NLP processing, fraud detection, reporting, and data storage modules.

A workflow-based testing approach is used to simulate realistic system usage. Rather than focusing exclusively on isolated components, testing emphasizes full end-to-end execution. The testing process begins with launching the Flask application locally and entering structured applicant data, including GPA, curriculum type, travel history, and risk factors. Essay input is processed through the NLP subsystem, and scoring outputs are validated for correctness.

Following this, PDF documents are uploaded to trigger fraud detection. Extracted signals and severity classifications are reviewed for consistency, and reviewer comments are added to both scoring and fraud outputs. Finally, reports are generated in PDF and TXT formats and validated for completeness. Database tables are then inspected to confirm correct persistence of analysis results and generated reports.

Although continuous integration (CI) pipelines are not currently implemented, automated unit testing has been introduced using the pytest framework. These tests focus on validating core fraud detection logic, feature extraction, severity classification, and structured output correctness. External dependencies such as OCR processing and AI API calls are isolated using mocking techniques via unittest.mock, ensuring deterministic test execution.

Automated testing was introduced in a later development stage (Sprint 5), while manual end-to-end workflow testing continues to validate full system behavior. Future improvements include expanding automated coverage to scoring, NLP processing, and report generation modules, as well as integrating CI pipelines such as GitHub Actions.

V.1.1. Unit Testing

Automated unit testing is implemented using the pytest framework to validate core system logic at the component level. Unit tests are primarily focused on the financial fraud detection and transcript fraud detection modules.

For the financial fraud detection module, unit tests validate helper functions, numerical parsing, date consistency checks, bank identity detection, line-item verification, and severity aggregation logic. For the transcript fraud detection module, tests validate transcript feature extraction, GPA detection, course code recognition, suspicious phrase detection, and document authenticity classification.

External dependencies such as OCR processing, PDF parsing, and AI-based analysis are isolated using unittest.mock. This ensures that unit tests are executed in a controlled environment without reliance on external services. These tests improve system reliability, maintainability, and regression resistance.

V.1.2. Integration Testing

Integration testing is conducted through execution of complete workflows within the Flask application to verify correct interaction between subsystems. This includes validating data flow between scoring, NLP analysis, fraud detection, comment handling, report generation, and database persistence modules.

Testing confirms that outputs generated by individual subsystems are correctly passed to downstream components, particularly from analysis modules into report generation. Reviewer comments are verified for correct propagation across the system and inclusion in exported reports. Database integration is validated by confirming consistent storage of analysis results and generated reports.

Future work includes implementing automated integration tests to simulate subsystem interactions using mock-based external service handling.

V.1.3. System Testing

System testing evaluates the International Student Scoring System as a complete integrated application. The objective is to ensure that the system operates correctly under realistic usage scenarios without runtime errors or inconsistencies.

Test scenarios include applicants with high and low scores, varying risk factor combinations, clean and suspicious financial documents, and multi-document fraud detection cases. Each scenario is executed through the full workflow, including input submission, scoring, fraud analysis, comment handling, and report generation.

System testing confirms that all subsystems function cohesively and that output remains consistent across the full pipeline.

V.1.4. Functional Testing

Functional testing ensures that all system requirements are correctly implemented and produce expected outputs. This includes verification of rubric-based scoring logic, risk deduction rules, NLP essay feature extraction, fraud severity labeling, comment persistence, export functionality (PDF and TXT), and database logging.

Each feature is manually tested against expected outcomes, with particular focus on ensuring that exported reports accurately reflect computed scores, deductions, fraud severity levels, and reviewer annotations.

V.1.5. Performance Testing

Formal performance benchmarking was not conducted; however, runtime performance was evaluated through observational testing during system execution. Key performance-sensitive components include OCR processing, PDF parsing, and external AI API calls.

To maintain acceptable performance, constraints such as document page limits were introduced to reduce processing time for large files. While performance was satisfactory under normal usage conditions, future work includes structured load testing, response time measurement, and concurrency evaluation.

V.1.6. Standards and Constraints Testing

Testing also validated compliance with system design constraints and security requirements. Authentication mechanisms were verified to use salted PBKDF2 hashing, and role-based access control was confirmed to restrict unauthorized operations.

Additionally, scoring consistency was validated against the predefined rubric, and fraud severity outputs were reviewed to ensure conservative and interpretable classifications. These checks ensure alignment with system security, reliability, and design constraints.

V.1.7. User Acceptance Testing

User acceptance testing was conducted through simulated usage by admissions staff personas. The evaluation focused on usability, clarity of scoring breakdowns, interpretability of fraud detection outputs, and ease of report generation.

Feedback during testing led to improvements in interface clarity, structured input design, and explanation of severity labels. The system was determined to be suitable for its intended decision-support workflow.

V.2. Environment Requirements

Testing of the International Student Scoring System was conducted in a local development environment using the Flask development server and a standard web browser interface.

The software environment includes Python 3.x, Flask, and SQLite configured in Write-Ahead Logging (WAL) mode. Additional dependencies include Hugging Face Transformers, Sentence Transformers, pdf2image, pytesseract, and ReportLab for NLP, OCR, and report generation functionalities.

The system operates as a standalone application and does not require distributed infrastructure for testing. However, internet connectivity is required for external AI API calls used in fraud detection. Due to usage costs associated with API requests, testing involving external services was conducted in a controlled manner.

Automated testing is implemented using pytest, with unittest. mock used for dependency isolation during unit testing. Manual workflow-based testing remains the primary method for validating full system behavior.

V.3. Test Results

Testing was conducted through structured end-to-end execution of the application using realistic applicant scenarios. Core workflows were executed multiple times to validate scoring accuracy, essay analysis consistency, fraud detection behavior, export functionality, and database persistence.

Rubric-based score calculations were verified using various GPA values and risk-factor combinations. Final composite scores reflected correct positive scoring and deduction logic. Essay submissions successfully generated structured NLP analysis results without runtime errors.

Fraud detection was tested using multiple uploaded PDF documents. OCR processing, heuristic checks, and AI-based anomaly detection executed successfully, producing document-level severity labels (LOW / MEDIUM / HIGH). Reviewer notes were correctly saved and displayed within the system.

Export functionality was validated by generating both PDF and TXT reports. Exported documents included final scores, deduction breakdowns, fraud severity labels, and all reviewer-entered comments. Database tables were inspected to confirm successful storage of analysis records and generated reports.

No critical failures or blocking defects were observed during testing. Under normal usage conditions, the system remained stable and responsive. Overall, the prototype meets its core functional requirements in its current state. Future improvements will focus on solidifying previous work, introducing automated testing and CI-based validation to further strengthen reliability as development continues.

VI. Projects and Tools used

This section summarizes the libraries, frameworks, and tools used in the development of the International Student Scoring System. These technologies collectively support backend processing, web interface development, machine learning integration, document analysis, and system deployment.

Tool/library/framework	Quick note on what it was for
Bootstrap	Generating layout and visual rendering on web interface
Python 3	Core programming language used to implement backend logic, scoring algorithms, fraud detection, NLP integration, and report generation.
Flask	Web framework used to build the application interface and manage routing between scoring, fraud detection, and reporting modules.
SQLite	Database used to store analysis records, reports, and application metadata.
Jinja2	Template engine used to dynamically render scoring results and fraud findings in the web interface.
HTML	Used to structure the web interface, including input forms and dashboards.
CSS	Used to style and organize the layout of the interface.
JavaScript	Used to handle interactive UI elements and export triggers.
Hugging Face Transformers	Used to load and run NLP models for structured essay evaluation.
PyTorch	Backend deep learning framework required to execute transformer-based NLP models.
OpenAI Python SDK	Used to perform AI-based analysis and structured evaluation tasks.
PyPDF	Used to parse and extract data from uploaded PDF documents.
OpenCV	Used for image-based heuristics and document anomaly detection
pytesseract	Used to perform OCR extraction from financial and transcript documents.

Pillow (PIL)	Used for image handling and preprocessing of uploaded document files.
ReportLab	Used to generate structured PDF export reports for scoring and fraud results.
Werkzeug Security	Used for secure password hashing and authentication handling.
Git	Used for version control and collaborative development.

Languages Used in Project			
Python	HTML5	CSS	JavaScript
SQL			

VII. Description of Final Prototype

The final prototype of the International Applicant Rating Algorithm (IARA) is a web-based decision-support system designed to assist admissions reviewers in evaluating international student applications. The system integrates structured rubric-based scoring, AI-powered essay analysis, financial and transcript fraud detection, and structured report generation into a unified interface. Rather than operating as separate tools, these components function together within a single workflow that allows reviewers to analyze applicant data, assess risk, and generate consolidated evaluation reports.

The prototype is implemented as a Flask web application with a modular backend architecture. The core functionality is divided into several subsystems, including an applicant scoring engine, an NLP-based essay evaluation module, a financial and transcript fraud detection module, a report generation and export component, and a database persistence layer. Each subsystem is implemented as an isolated service layer, while the Flask application coordinates communication between them.

The system allows admissions reviewers to input structured applicant information such as GPA, curriculum type, travel history, and risk indicators, while also submitting personal essay text for automated analysis. Supporting financial and transcript documents can be uploaded for fraud screening. Reviewers are able to analyze results, add structured comments, and export consolidated reports that include scoring breakdowns, fraud severity labels, and reviewer notes. All results are stored in the database to maintain consistency and traceability across evaluations.

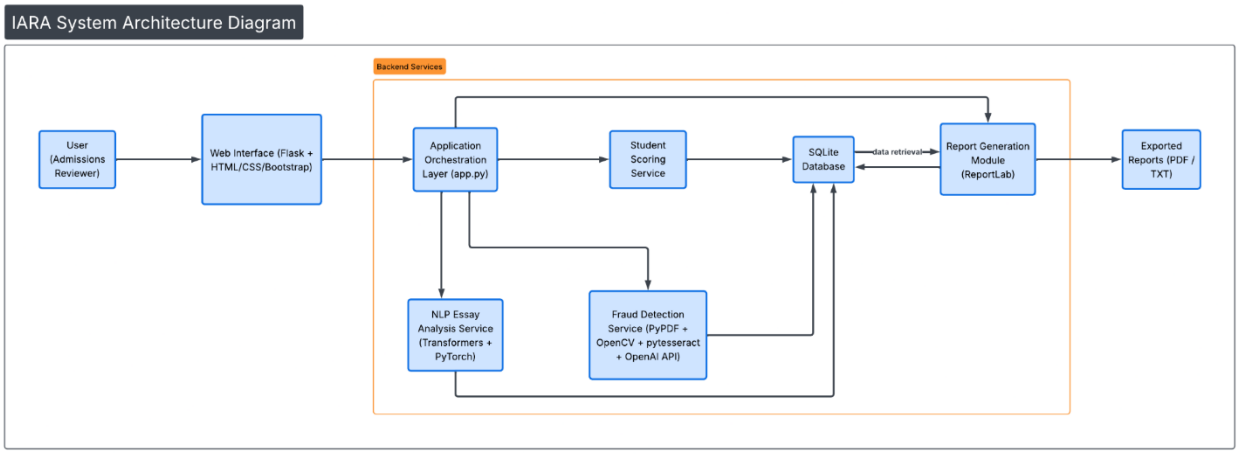

Figure 7: System Architecture Diagram

This figure illustrates the high-level architecture of the IARA prototype and the flow of data between its major components. User interactions begin at the web interface, which routes requests through the application orchestration layer (app.py). The orchestration layer coordinates communication between the student scoring service, NLP essay analysis service, fraud detection service, report generation module, and the SQLite persistence layer. Processed results are stored in the database and may be exported as structured PDF or TXT reports for admissions review.

- **User Interface Workflow**

The user interface is designed to provide admissions reviewers with a structured and guided evaluation workflow. Upon logging in, the reviewer is presented with the primary scoring dashboard, where structured applicant information can be entered. This includes student ID, country, admissions GPA, curriculum type, travel history, selected risk indicators, and personal essay text. The form uses controlled inputs such as dropdown menus and checkboxes to maintain consistency in scoring logic and prevent malformed data entry. Once the reviewer selects “Analyze Student,” the system processes the inputs and displays a breakdown of the positive score, negative deductions, and final composite score within the results panel.

Essay analysis is integrated directly into this workflow. When essay text is submitted, the NLP subsystem evaluates structure, tone, and language consistency using transformer-based models. The generated metrics are incorporated into the scoring output and displayed alongside other evaluation factors. Reviewers may add comments directly within the interface to document observations, clarify concerns, or provide additional context for decision-making.

The system also includes a dedicated document screening interface for financial and transcript uploads. Reviewers can upload one or more PDF files, which are then processed through OCR extraction, image-based heuristics, and AI-assisted anomaly detection. Page-level findings are displayed, and each document receives a consolidated severity classification of LOW, MEDIUM, or HIGH. Reviewer notes can be entered beneath each document’s results. After completing the evaluation, scoring results and fraud findings can be exported as structured PDF or TXT reports. All generated outputs and associated metadata are stored in the database to ensure consistency, traceability, and future reference.

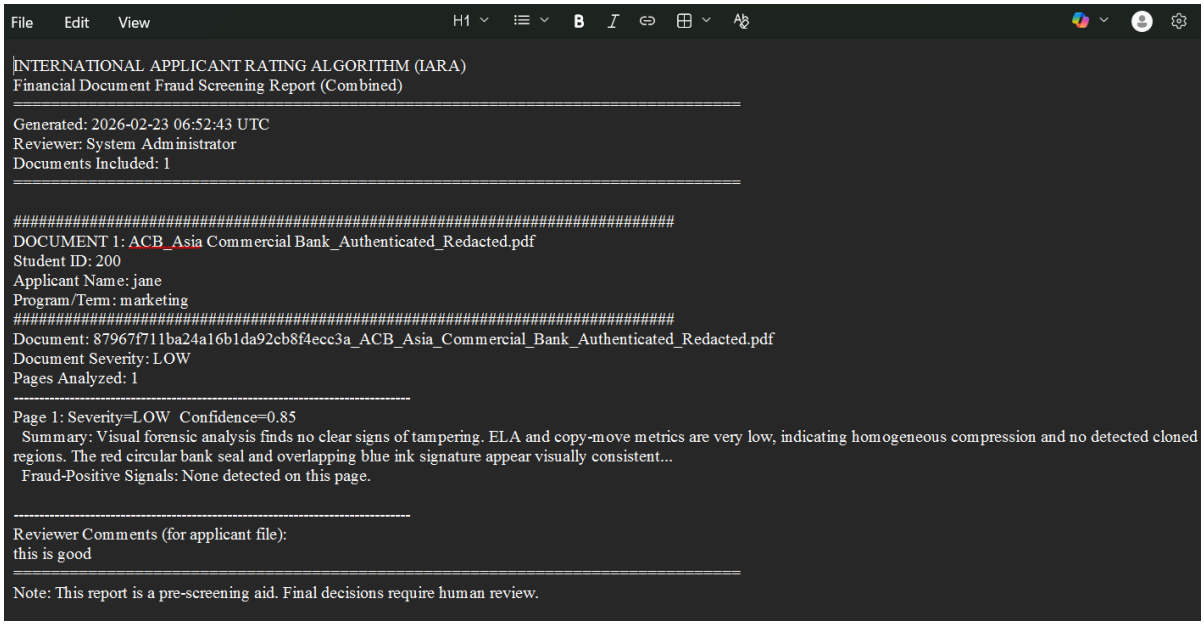

Figure 8: Generated Report

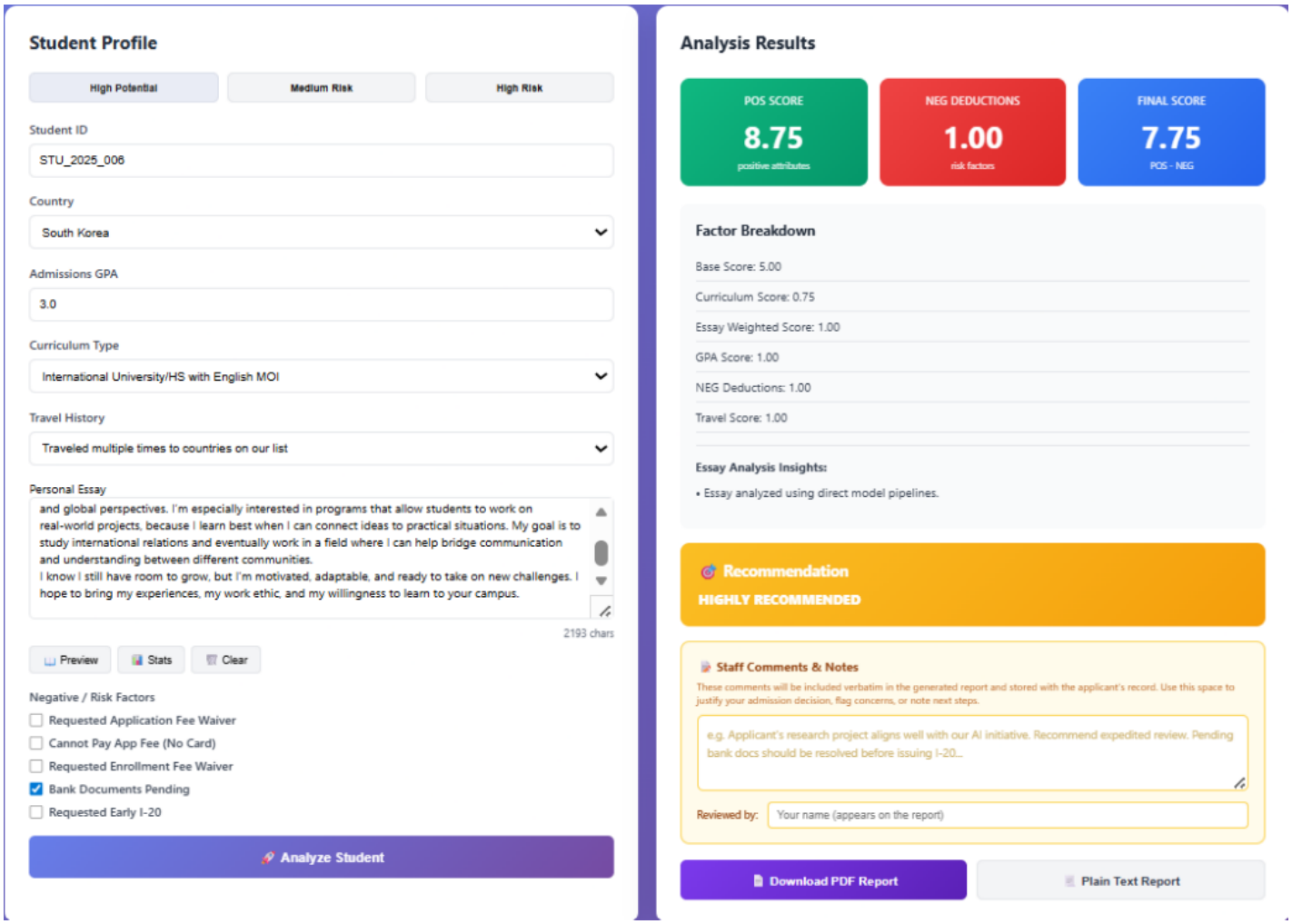

Figure 9: Primary Scoring Dashboard

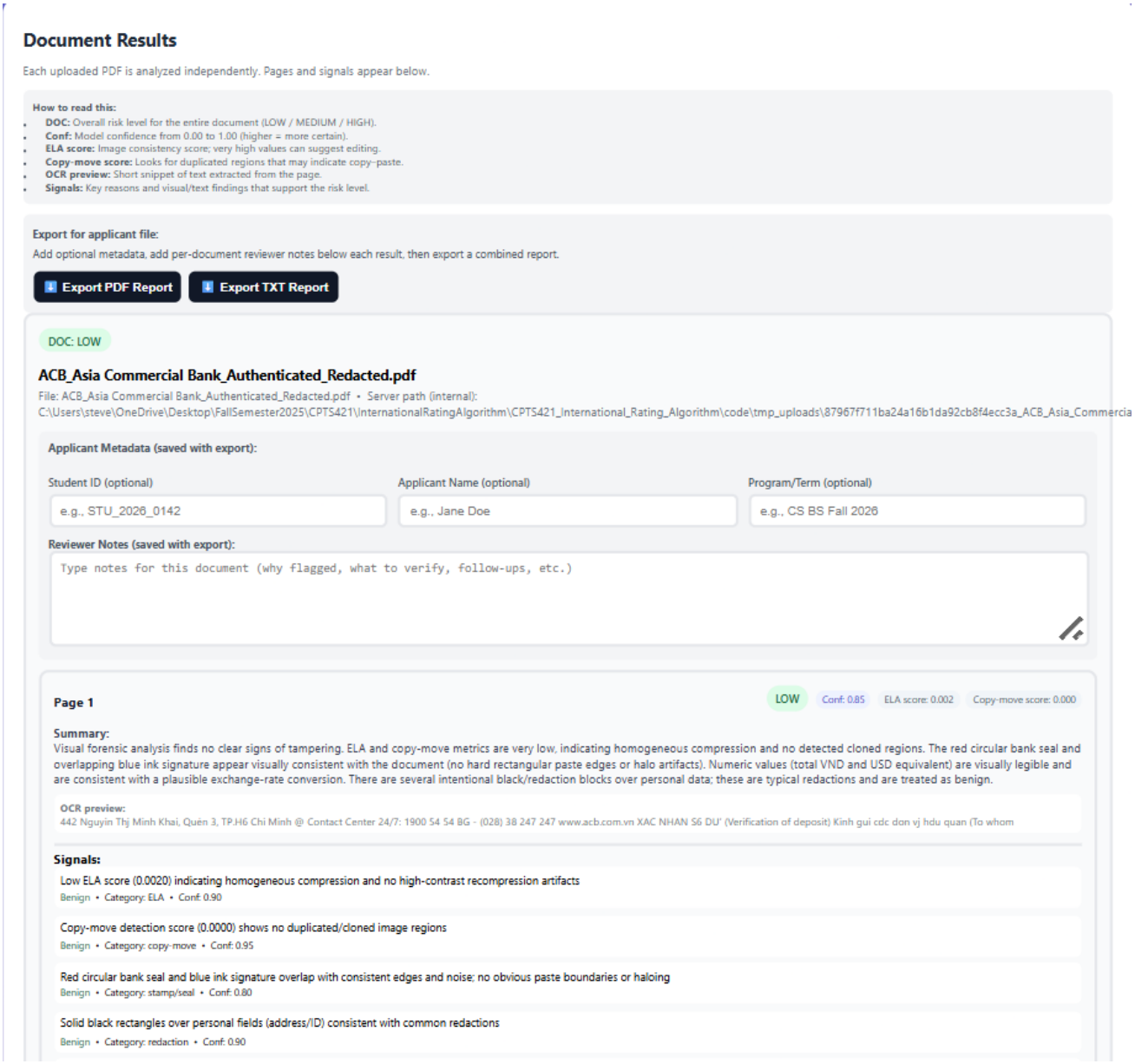

Figure 10: Document Result Page

- **Major Use Case Scenarios**

The primary use case of the IARA prototype involves a standard applicant evaluation workflow. An admissions reviewer logs into the system and enters structured applicant data, including GPA, curriculum type, travel history, and selected risk indicators. The reviewer submits the applicant's personal essay, which triggers automated NLP-based evaluation.

The system computes a final score using rubric-based logic, applies any risk deductions, and generates a recommendation category. The reviewer may then add structured comments before exporting the complete evaluation report.

A second use case focuses on financial and transcript fraud screening. The reviewer uploads one or more supporting documents in PDF format. The system performs OCR extraction, image-level heuristic analysis, and AI-assisted anomaly detection. Page-level findings are displayed within the interface, and the system assigns a consolidated severity classification (LOW, MEDIUM, or HIGH). Reviewer notes can be added to the fraud analysis results before exporting findings as part of the final report.

A third use case represents a full end-to-end evaluation. The reviewer enters applicant data, submits the essay, uploads supporting documents, reviews scoring and fraud findings, adds comments, and exports a consolidated PDF report. The system stores all analysis records and generated reports in the database to ensure traceability and consistency. This scenario demonstrates complete subsystem orchestration, from structured input to final report generation.

- **Prototype Capabilities and Current Status**

The current IARA prototype functions as a complete decision-support system for evaluating international applicants. It integrates structured rubric-based scoring, AI-powered essay analysis, financial and transcript fraud detection, reviewer comment handling, report generation, and database storage into a single working application. All major subsystems are coordinated through the Flask orchestration layer, allowing the system to execute the full evaluation workflow from structured input to exported report.

The prototype runs reliably in a local deployment environment and demonstrates full end-to-end functionality across scoring, NLP evaluation, fraud screening, and reporting. While there is room for additional testing automation and future expansion, the system in its current state successfully demonstrates the intended workflow and core analytical capabilities of the International Applicant Rating Algorithm.

VIII. Social Responsibility and Broader Impacts

The International Student Scoring System was developed with ethical, legal, and professional considerations due to its use in supporting admissions decision-making. Since the system processes sensitive data such as academic records, essays, and financial documents, emphasis was placed on privacy, fairness, and responsible AI use throughout the design process.

A key ethical decision was to ensure the system functions strictly as a decision-support tool rather than an automated decision-maker. While the system produces scores, fraud classifications, and recommendations, final decisions remain under human reviewer control. This aligns with professional responsibilities in software engineering, particularly in high-stakes domains where accountability and fairness are critical.

Data privacy and security were also prioritized. The system uses salted PBKDF2 hashing for authentication and role-based access control to protect sensitive functionality. Document processing is performed locally where possible, and reliance on external AI services is minimized to reduce exposure of personal data.

A limitation of the system is the use of external AI APIs for fraud detection, which introduces third-party data handling risks. To mitigate this, only necessary data is transmitted, and human oversight is maintained for all high-risk outputs. Overall, the design aligns with professional ethical standards, emphasizing transparency, accountability, and responsible use of AI in decision-support systems.

In terms of broader impacts, the system aligns with WSU EECS goals by addressing the problem of inconsistent and inefficient international admissions evaluation. It improves this process through a structured, explainable workflow combining rubric-based scoring, NLP essay analysis, and fraud detection.

The interface is designed for non-technical users using structured inputs and clear severity labels (LOW / MEDIUM / HIGH), improving accessibility and usability. Although external user testing was limited, internal feedback helped refine the system's usability and clarity.

The project also supports educational and research goals by providing a modular foundation for future work in AI-assisted decision systems and document analysis. Its documented and structured design enables reproducibility and potential future expansion in both academic and real-world settings.

V. Product Delivery Status

The International Student Scoring System was completed and demonstrated during the final phase of the Spring 2026 project cycle. The system was presented to the course instructor for evaluation and is fully functional as a local web-based application.

The source code is stored in a GitHub repository, which includes the complete Flask application, database schema (SQLite), NLP and fraud detection modules, and report generation components. The repository also contains setup instructions required to rebuild and run the system.

No physical equipment was required for the project. All development and testing were conducted on local machines. The system can be executed by installing the required dependencies, configuring environment variables (including external AI API keys), initializing the database, and running the Flask server. The project is fully reproducible from source code, and all essential materials required for deployment are included in the repository.

VI. Conclusions and Future Work

1. Limitations and Recommendations

Although the IARA prototype demonstrates full end-to-end functionality, several limitations remain. A key limitation is latency in the fraud detection subsystem, caused by OCR processing and external AI API calls. While results remain accurate, response time is affected. This could be improved through asynchronous processing, background task execution, and non-blocking progress indicators.

Another limitation is the export functionality report. Although PDF and TXT outputs are generated successfully, formatting and customization are limited. Future improvements could include enhanced layout design, institutional branding templates, and configurable export options.

The system also relies primarily on manual workflow testing, with limited automated unit or integration tests. While the modular design supports testability, formal test suites were not fully implemented. Adding automated testing for scoring logic and subsystem integration would improve reliability.

Finally, the recommendation system uses fixed score thresholds (e.g., Admit, Review, Reject), which may require adjustment over time based on stakeholder feedback. However, the modular design allows these thresholds to be updated without major system changes.

2. Future Work

Future improvements of the IARA system focus on scalability, robustness, and usability. Fraud detection can be enhanced by integrating additional heuristic methods and alternative AI models, while performance can be improved through asynchronous processing, caching, and pipeline optimization.

The scoring and recommendation framework can also be refined in collaboration with stakeholders to align with evolving admissions requirements. The modular structure allows flexible updates to scoring rules without system redesign.

From a software engineering perspective, adding automated testing, CI/CD pipelines, logging, and monitoring would significantly improve system reliability and maintainability.

Finally, the system could be extended into a scalable institutional platform integrated with admissions systems, providing a foundation for future research, enhancements, and real-world deployment.

VII. Acknowledgements

We would like to sincerely thank our client, Trevor Kingsley, for his guidance and constructive feedback throughout the project. Special thanks are also extended to our faculty mentor, Dr. Parteek Kumar, for continuous support and valuable insights during development. We further acknowledge the contributions of all team members for their collaboration in design, implementation, and testing. Finally, we recognize the use of tools and technologies such as Python, SQLite, and Hugging Face models that made this project possible.

VIII. Glossary

ACM Code of Ethics - Professional guidelines for computing professionals emphasizing fairness, transparency, and responsibility.

Applicant Scoring Module - Calculates academic, financial, and behavioral scores for applicants using GPA, curriculum, essays, and risk factors.

Composite Score - The final applicant rating combining positive points, negative deductions, and AI-driven insights.

CI (Continuous Integration) - Development practice where code changes are regularly merged and tested automatically.

Data Layer - Responsible for storing and retrieving data; currently file-based and in-memory, with a database planned.

ELA (Error Level Analysis) - Image forensics technique highlighting areas of possible tampering.

Essay Analysis Module - Evaluates essays using NLP models for clarity, tone, structure, and relevance.

Export Subsystem - Generates structured output files (CSV/XLSX) from evaluation results.

Fraud Detection Module - Analyzes financial documents and transcripts via OCR, heuristics, and GPT-based reasoning.

GDPR - EU regulation governing privacy and user control over personal data.

Hugging Face - Platform providing pretrained NLP models for essay analysis.

NLP (Natural Language Processing)- AI field for understanding and analyzing human language.

OCR (Optical Character Recognition)- Converts text in images or PDFs into machine-readable text.

Risk Factors / Risk Deduction Logic - Conditions that reduce an applicant's score, e.g., missing documents, travel issues.

RBAC (Role-Based Access Control) - Restricts actions based on user roles (admin, counselor).

Slate - WSU's CRM system for admissions; future IARA versions integrate via export/API.

UAT (User Acceptance Testing) - Conducted by end users (counselors) to confirm system readiness.

WCAG 2.2 - Accessibility guidelines for web content to aid users with disabilities.

IX. References

[1] IEEE, *IEEE Standard 830: Software Requirements Specification*, 1998.

[2] ISO/IEC, *ISO/IEC 25010:2011 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE)*, 2011.

[3] S. Keppler and C. Snyder, "Teachers feel most productive when they use AI for teaching strategies," University of Michigan, 2025. [Online]. Available: https://umich.edu/research/ai-teaching-strategies

[4] Client Rubric, *Internal documentation for IARA development*, 2025.

[5] GenAI Brainstorming, *Internal team session notes*, 2025.

[6] Family Educational Rights and Privacy Act (FERPA), "34 CFR Part 99 Family Educational Rights and Privacy," U.S. Department of Education, accessed Dec. 2, 2025. [Online]. Available: https://studentprivacy.ed.gov/ferpa

[7] General Data Protection Regulation (GDPR), *Official Journal of the European Union*, 2018. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[8] Hugging Face, *Transformers Documentation*, 2025. [Online]. Available: https://huggingface.co/docs/transformers/index

[9] OpenCV, "OpenCV-Python Tutorials: Introduction to OpenCV," OpenCV Documentation, version 4.x, 2025. [Online]. Available: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

[10] OpenAI, "OpenAI Multimodal API Documentation," OpenAI Documentation, 2025. [Online]. Available: https://platform.openai.com/docs/guides/multimodal

[11] Flask Documentation, "Flask Web Framework Documentation," 2025. [Online]. Available: https://flask.palletsprojects.com/en/latest/

X. Appendix A – Team Information

1. Khushi Panchal

Khushi Panchal is a senior at Washington State University, graduating in Spring 2026 with a Bachelor of Science in Computer Science. She is passionate about data analytics, web development, and software engineering. Khushi has hands-on experience with academic and personal projects, including full-stack web applications and data-driven dashboards. Her technical skills include Python, C++, JavaScript/TypeScript, HTML/CSS, and familiarity with cloud platforms such as AWS and Google Cloud. For the International Applicant Rating Algorithm project, Khushi led the development of the batch-processing analytical dashboard, implemented CSV import/export functionality, and authored comprehensive project documentation.

2. Steven Bennett

Steven Bennett is a Software Engineering student interested in AI-assisted systems, backend development, and applied machine learning. He has worked on various school and personal projects and is developing strong experience in building full-stack prototypes. Steven's technical skills include Python, Flask, SQL, Java, and OpenAI-powered reasoning tools, along with hands-on experience implementing OCR and computer-vision pipelines. For this project, his responsibilities include developing the Financial and Transcript Fraud Detection subsystems, implementing associated Flask routes and UI outputs, coordinating communication with the project stakeholder, and contributing to system integration and part-time team leadership.

XI. Appendix B - Example Testing Strategy Reporting

1. **Requirements Being Tested:**

- API Functionality: Verified that all endpoints (/api/analyze, /api/batch-analyze, feedback submission, and role-based login) perform as expected with valid and invalid inputs.
- User Access & Security: Tested role-based login to ensure proper access control. Verified password security using SHA-256 hashing.
- Data Handling & Scoring: Confirmed that essay analysis, score computation, and similarity analysis produce expected outputs.
- UI Components: Tested feedback text box and dashboard updates to ensure smooth interaction and accurate data display.
- File Handling & API Improvements: Ensured file restructuring and API enhancements support consistent request handling.

2. Testing Tools & Evidence

- Postman (Free): Used for testing RESTful API endpoints, request validation, batch operations, and error handling. Postman collections were created to automate repeated tests and document expected responses. Screenshots of successful requests, error responses, and role-based access tests are included below.
- Unit Testing: Python scripts were used to test critical functions in the scoring and analysis modules.
- Browser Testing: Checked dashboard functionality across Chrome and Edge to ensure UI elements like the feedback text box render correctly.

3. User Testing & Feedback

- User Survey: Collected feedback from team members and potential end-users regarding the clarity of the dashboard, ease of submitting feedback, and usability of analysis results.
- Quotes / Feedback Examples:
    - “The role-based login makes it easy to differentiate access between admins and reviewers.”
    - “The feedback text box allows quick suggestions for each student’s analysis.”
- Discussion: Users found the batch analysis API highly useful, though some requested clearer error messages for invalid inputs. These observations will inform future improvements.

XII. Appendix C - Project Management

The International Applicant Rating Algorithm team followed a structured but flexible project management approach throughout development. While formal weekly meetings were not always scheduled, team coordination occurred consistently through Microsoft Teams communication, shared documentation, and GitHub project tracking tools.

- **Meetings with Instructor**

The team met formally with the course instructor on February 19th to review project progress, system architecture, and stakeholder communication. During this meeting, we discussed implementation decisions, received feedback on technical design, and clarified deliverable expectations. The instructor also monitored our progress throughout the semester by reviewing repository activity and confirming that stakeholder communication was ongoing.

- **Meetings with Stakeholder**

Meetings with the stakeholder (Trevor Kingsley) occurred periodically rather than on a strict weekly schedule. These meetings were primarily focused on validating system functionality, identifying areas for improvement, and gathering feedback on scoring logic and fraud detection outputs. When direct meetings were not scheduled, communication occurred through Microsoft Teams messaging to clarify requirements and confirm alignment with admissions goals.

The typical agenda for stakeholder discussions included:

- Demonstrating current system features
- Reviewing scoring outputs and fraud detection results
- Asking for clarification on admissions policies
- Identifying potential improvements or additional features

These interactions were critical in ensuring the prototype remained aligned with real-world institutional needs.

- **Internal Team Coordination**

Team coordination was maintained through Microsoft Teams chat, shared notes, and GitHub project management tools. We used a GitHub Kanban board to track features, bug fixes, and sprint tasks. GitHub Issues were created to document feature implementation, debugging tasks, and enhancements. Progress was monitored by updating issue statuses and referencing commits tied to each task.

Meeting notes and design discussions were documented and shared within Teams to maintain transparency and ensure both team members remained aligned. This asynchronous

communication approach allowed flexibility while still maintaining accountability.

- **Planning Documents and Tools Used**

Project planning and coordination were supported through:

- GitHub repository and version control
- GitHub Issues for task tracking
- GitHub Kanban board for sprint organization
- Microsoft Teams for communication and note sharing
- Email communication when necessary

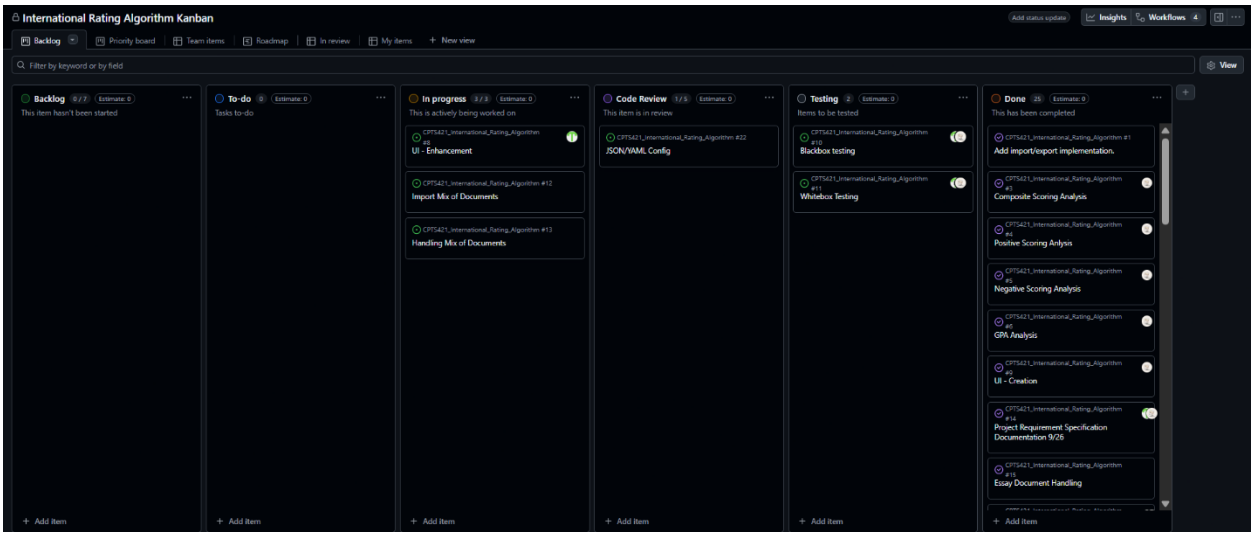

Figure 11: Kanban Board

- **Most Beneficial Activities**

The most beneficial project management activities were stakeholder feedback sessions and GitHub tracking. Stakeholder discussions ensured the system remained grounded in real admissions requirements, while GitHub tracking tools provided visibility into development progress and task ownership. The combination of structured issue tracking and consistent communication allowed the team to adapt quickly and maintain steady forward progress.